

Retrieval-Augmented Generation and Embeddings

Sample educational document created for testing IntelliAssist AI.

1. What are Embeddings?

An embedding is a numerical representation of information in a vector space. For text, embedding models transform words, sentences, paragraphs, or documents into vectors that capture semantic relationships. Similar meanings should produce vectors that are relatively close under a suitable similarity measure. Embeddings are therefore useful for semantic retrieval rather than relying only on exact keyword matches.

2. Vector Databases

A vector database or vector index stores embeddings and supports similarity search. FAISS, for example, provides efficient similarity-search functionality for dense vectors. A document application can store each chunk together with metadata such as the document name and page number. When a user asks a question, its embedding is compared with stored embeddings to identify relevant chunks.

3. Semantic Search

Traditional keyword search primarily looks for matching terms. Semantic search attempts to find information with related meaning even when the exact words differ. For example, a query about 'methods for reducing model errors' may retrieve a passage discussing regularization and generalization even if those exact query words are not present.

4. What is RAG?

Retrieval-Augmented Generation (RAG) combines information retrieval with generative language models. Instead of asking a language model to answer only from its learned parameters, the application first retrieves relevant external context and places that context into the model prompt. The model then generates an answer grounded in the retrieved material.

5. RAG Pipeline

A typical RAG pipeline has two phases. During indexing, documents are loaded, text is extracted, chunks are created, embeddings are generated, and vectors plus metadata are stored. During querying, the user's question is embedded, similar chunks are retrieved, the retrieved context is combined with instructions, and a language model generates the response. Source metadata can then be displayed to the user.

6. Why RAG is Useful

RAG is useful when information is private, frequently changing, domain-specific, or too large to place directly into every prompt. It can reduce the need for model fine-tuning for many document-question-answering applications. It also provides a mechanism for displaying the retrieved evidence behind an answer.

7. Limitations of RAG

RAG quality depends on both retrieval and generation. If the correct passage is not retrieved, the language model may not have enough evidence to answer accurately. Poor chunking, weak embeddings, noisy documents, or an unsuitable number of retrieved passages can reduce performance. Even with good retrieval, a language model can misinterpret context. Evaluation should therefore test retrieval relevance, answer correctness, and citation accuracy.

8. RAG in IntelliAssist AI

IntelliAssist AI follows the RAG architecture described in the project brief. Documents are uploaded, text is extracted and preprocessed, embeddings are generated and stored in a vector database, and a LangChain-based RAG pipeline retrieves relevant context before generating answers. The interface can display source citations alongside the response.